

UNIVERSITÉ LIBRE DE BRUXELLES



SSD Project Report: Dashcam

INFO-Y115 - Secure software design and web security

GROUP MEMBERS

Omar Gueriri
Kemayou Tchakounte Brank Aubin
Adrien de Suray
Aymane Taifour
Grari Mohammed Achraf
Fernandes do Rosario Tiago
Manuelle Ndamtang
Patrick Preda
Mickael Kovel
Adewuyi Samuel Dan
Md Ziauddin
Simon Peetroons
Kishore Prakash
Vitaly Belvine Djonfang Tchuangue
Othmane Tabiane
Senisi Lobo Marco

January 19, 2025

Contents

1	Introduction	1
2	Structure	1
2.1	Client	1
2.2	Server	1
3	Security	2
3.1	Key Management	2
3.2	Encryption and Decryption	2
3.3	Logging and Monitoring	2
4	Additional Questions	3

1 Introduction

In the scope of our course, we were tasked with building a secure application that allows the selective sharing of dashcam footage with relevant authorities. To this end, we built an application with a client/server architecture leveraging Next.js and Node.js.

This report has the goal of clarifying some of the choices made during the development of this application. As such in this report we will start by succinctly describing the structure of the application without forgetting to mention the benefits of using Next and Node.

2 Structure

We developed the dashcam app that allows regular users to share videos with trusted users. The app has a client/server architecture allowing the client application to connect to the server to store or retrieve videos. The videos a user has access to are videos they own or videos that were shared with them.

This is followed by a brief description of the most relevant security aspects of the project. In including key handling and encryption management.

We end this report with the answers to additional questions relevant to security that would allow a better understanding of the functioning of the project.

2.1 Client

The client was built using Next.js giving access to a multitude of tools and libraries that speed up development. Here is a short list of the pros of using Next.js:

1. **Server-Side Rendering (SSR):** Improves SEO and performance by pre-rendering pages on the server.
2. **Static Site Generation (SSG):** Generates static pages at build time for faster performance.
3. **API Routes:** Provides built-in serverless API support for backend logic.
4. **Automatic Code Splitting:** Loads only the required JavaScript for each page.
5. **Integrated Routing:** Simplifies routing using a file-based structure.
6. **Hot reloading:** When a change is made it is automatically updated speeding up development.
7. **Input sanitization:** Sanitize user inputs automatically
8. **Protection against XSS attacks:** Next.js automatically applies proper encoding and escaping of data when rendering it within HTML or JavaScript tags. This effectively limits common XSS attack vectors.

2.2 Server

The server was built using Node.js giving access to a multitude of tools and libraries that help bolster security. Here is a short list of the pros of using Node.js:

1. **Event-Driven Architecture:** Efficiently handles multiple simultaneous requests with non-blocking I/O.

2. **Scalable:** Supports high traffic with lightweight, asynchronous processing.
3. **Full-Stack JavaScript:** Allows unified development with the same language on client and server.
4. **Rich NPM Ecosystem:** Access to thousands of reusable packages and modules.
5. **Real-Time Applications:** Ideal for apps like chats or collaborative tools.
6. **Cross-Platform Compatibility:** Runs seamlessly on Windows, macOS, and Linux.

When used together, Node.js and Next.js offer a powerful full-stack solution for modern web applications, enabling unified development with JavaScript and seamless deployment across various platforms.

3 Security

3.1 Key Management

During account creation, and account usage multiple cryptographic keys are generated to secure the user's data:

- **Symmetric Key:** Used to encrypt videos uploaded by the user.
- **Public Key:** Used to encrypt sensitive information sent to the user.
- **Private Key:** Used by the user to decrypt sensitive information intended for them.

Notably, new symmetric keys are generated for each video the user encrypts and sends to the server, ensuring individual video security.

3.2 Encryption and Decryption

To maintain robust security, all sensitive data is encrypted and decrypted client-side. The encryption mechanisms are implemented using built-in Next.js functions:

- **RSA Encryption:** Used for the public/private key pair to secure sensitive information.
- **AES-GCM Encryption:** Used for encrypting and decrypting videos with their respective symmetric keys, ensuring confidentiality and integrity.

3.3 Logging and Monitoring

An automated logging system tracks and verifies all actions that could potentially compromise application security. This ensures accountability and facilitates swift responses to security threats. The logging system is implemented using Winston, a robust and flexible logging library. It has been configured to handle four distinct log levels:

- **INFO:** Used for standard log outputs and general system activities.
- **WARN:** Indicates important information or situations that require attention.
- **ERROR:** Logs critical issues and failures that may require immediate resolution.
- **DEBUG:** Reserved for development and testing purposes to aid in debugging.

Sanitization

Every log entry undergoes a thorough sanitization process to ensure security and consistency. This step is crucial to prevent potential issues arising from improperly formatted or malicious log data. The sanitization process includes:

- Preventing log injection attacks by removing newlines and special characters.
- Resolving formatting issues and replacing problematic characters (e.g.,
- Stripping out HTML/JavaScript tags to avoid embedding malicious content.
- Obfuscating sensitive data (e.g., passwords, tokens, keys) to protect user information.
- Truncating excessively long messages to a manageable length.

This proactive measure ensures logs remain clean, secure, and reliable, even as the system grows and more contributors join the project. By addressing these potential issues early, we minimize the risks of introducing unwanted content into logs in the future.

Hashing

To ensure the integrity of the log entries, each log is stored alongside its hash. When verifying the integrity of the log, the hash is computed once again and compared with the hash stored.

Log Analysis

The logging system incorporates an analysis component to detect abnormal or unwanted behaviour within the system. This feature proactively scans logs for patterns or anomalies, including:

- Frequent video delays, which may indicate performance or network issues.
- Endpoint errors, which help identify malfunctioning APIs or services.
- Suspicious user activities, which could signal security threats or misuse.

Currently, issues detected in the logs are simply printed. However, with more time, we can imagine a system sending email or SMS to the administrator team and alerting them about the issue raised.

4 Additional Questions

- Do I properly ensure confidentiality?
 - Are sensitive data transmitted and stored properly?
Videos, logs, and user information are stored in the database, while the access token is stored locally on the client side since the server is considered untrusted.
Logs and the user's username are not encrypted, though logs are associated with a hash to ensure their integrity. Videos and other user information are encrypted prior to being stored in the database.
 - Are sensitive requests sent to the server transmitted securely? The communication between clients and the server is securely transmitted via HTTPS, ensuring data encryption and protection against interception. However, the certificates are encrypted but not provided by a trusted certificate authority. With more time, this security can be further by obtaining one from a reliable certificate authority.

- Does a system administrator have the ability to access any sensitive data?
The server administrator has access to videos, logs, and user information. However, videos and user information (excluding the username) are encrypted, and the server administrator cannot decrypt them. Logs have an associated hash to ensure their integrity.
- Did I harden my authentication scheme?
 - Do I use Captcha, MFA, or a zero-knowledge proof scheme?
After registration, a Time-Based One-Time Password (TOTP) code is generated to enable two-factor authentication (2FA) using Google Authenticator. A QR code is provided to link the account with Google Authenticator, which generates a 30-second OTP for secure login.
- Do I properly ensure the integrity of stored data?
Logs are associated with a hash to ensure their integrity, allowing any unauthorized modifications to be detected. Videos and user information are stored in encrypted form, ensuring that any tampering will result in a failed decryption, effectively protecting data integrity.
 - Do I properly ensure the integrity of sequences of items?
No. Any changes made are logged and will be visible for review. Additionally, as mentioned above, any modifications to videos, logs, or user information will be detected through associated hashes (for logs) or failed decryption (for videos and user information).
- Do I properly ensure non-repudiation?
Yes, a robust logging system with associated hashes ensures non-repudiation. Each action performed within the system is logged and secured with a hash, providing verifiable evidence of modifications. This makes it impossible to deny actions that have been recorded.
- Do my security features rely on secrecy beyond cryptographic keys and access codes?
Yes, the system relies on secrecy for the `ACCESS_TOKEN_SECRET`, which is used to generate tokens and hashes for logs. This element of secrecy complements cryptographic keys and access codes.
- Am I vulnerable to injection attacks?
 - URL, SQL, JavaScript, and dedicated parser injections.

URL: The server validates all incoming URLs and redirects users to the home page if any irregularities are detected. SQL: Since MongoDB is a NoSQL database, the system is not vulnerable to traditional SQL injection attacks. However, inputs are still validated and sanitized before being stored to ensure security. JavaScript: Inputs are sanitized to prevent JavaScript injection attacks. Additionally, Next.js automatically applies proper encoding and escaping of data when rendering it within HTML or JavaScript tags, effectively limiting common XSS attack vectors.

Logs: As explained in the logging section, logs are sanitized to prevent overly long messages, exposure of sensitive information, or other potential vulnerabilities.
- Am I vulnerable to data remanence attacks?
Yes, deleted videos are removed at the user's request. However, these videos could still be subject to data remanence attacks, as their remnants may persist in storage. This issue could be mitigated by implementing a secure deletion mechanism that overwrites the memory addresses where the video chunks were stored.

- Am I vulnerable to fraudulent request forgery? Yes, a malicious actor could potentially forge a new video chunk. However, when the forged chunk is decrypted, it will trigger an issue due to the mismatch between the encrypted data and the decryption process. While this mechanism helps detect fraudulent requests and prevent unauthorized data from being accepted, it does not provide a robust defence against such attacks and would require additional measures to fully mitigate the risk.
- Am I monitoring enough user activity to detect malicious intents or analyze an attack a posteriori?
Yes, our logging system enables the detection of suspicious activities, such as frequent video delays, excessive user actions within a short period, or other anomalous behaviour patterns.
 - Am I properly sanitizing user input?
Yes, input sanitization is implemented for both the email and country fields. The country selection is fixed, meaning no new input is allowed, and the email input is validated for proper formatting. Additionally, since we are using Next.js, it automatically handles input sanitization, protecting against injection attacks and other potential vulnerabilities associated with improper sanitization.
 - Did I implement some form of anomaly detection?
Yes, we implement anomaly detection by scanning logs for irregularities. These anomalies could include endpoint errors, malfunctioning APIs, or unusual user activity patterns, such as abnormal login attempts or unexpected behaviour during video interactions.
 - Do I use a whistleblower client?
No, we do not use a whistleblower client. Suspicious activities are detected through server-side logs, not by a client-side mechanism.
- Am I using components with known vulnerabilities? We are using HTTPS for secure communication and rely on a free solution to generate our SSL certificates. However, as these certificates are not issued by a recognized certificate authority (CA), there could be potential trust issues. To mitigate this risk, we could enhance security by using SSL certificates from a reputable CA.
- Is my system updated?
Yes, we ensure that we are using the latest versions of all the tools and components in our system.
- Is my access control broken? (cf. OWASP Top 10)
No, access control is properly implemented. Disconnected users attempting to access restricted pages are automatically redirected to the homepage. Only authenticated users with valid sessions have access to the service.
- Is my authentication broken? (cf. OWASP Top 10)
No, authentication is secure. We use Two-Factor Authentication (2FA) with Google Authenticator. To log in, the user must enter the One-Time Password (OTP) generated by their Google Authenticator app. This code is refreshed every 30 seconds, ensuring that the authentication process remains secure.
- Are my general security features misconfigured? (cf. OWASP Top 10)
While we are using HTTPS for secure communication, we rely on a free solution to generate our SSL certificates. However, these certificates are not issued by a reputable certificate authority (CA). This could potentially lead to trust issues, as browsers and other clients may flag the connection as insecure due to the lack of a trusted CA.